



计算机系统（2）实践手册

作者：ACM 班课程助教

组织：ACM 班，上海交通大学

时间：2024-2025 春季学期

版本：1.1.1

目录

第 1 章 总述	1
1.1 作业内容	1
1.2 作业安排（初步）	1
第 2 章 项目环境准备	3
2.1 基准代码	3
2.2 在 QEMU 中启动 Linux 镜像	3
2.2.1 QEMU 安装（二选一）	3
2.2.1.1 软件包安装	3
2.2.1.2 源码编译安装	3
2.2.1.3 常见依赖问题	3
2.2.2 Linux 内核编译	4
2.2.3 BusyBox	4
2.2.3.1 直接安装	4
2.2.3.2 手动编译安装	4
2.2.4 EDK2 编译	5
2.2.5 制作 initramfs	5
2.2.5.1 创建启动脚本	5
2.2.5.2 打包文件系统	5
2.2.6 启动 QEMU	5
2.2.6.1 图形界面启动	5
2.2.6.2 字符界面启动	6
2.2.7 验证	6
2.3 Linux 环境部署 EDK2 开发环境	6
2.3.0.1 下载源码	6
2.3.0.2 网上找到的可能需要安装的一堆东西	6
2.3.0.3 make 一下 BaseTools	6
2.3.0.4 编译 UEFI 模拟器和 UEFI 工程	7
2.3.0.5 运行	7
第 3 章 系统启动	8
3.1 Read ACPI Table	8
3.2 Hack ACPI Table	8
3.3 UEFI 运行时服务	9
第 4 章 特权态与系统调用	10
4.1 用户定义的系统调用	10
4.2 vDSO	10
4.3 无需中断的系统调用	11
第 5 章 内存管理	12
5.1 页表和文件页	12
5.2 内存文件系统	12

5.3 内存压力导向的内存管理	12
第 6 章 文件系统	13
6.1 inode 和扩展属性管理	13
6.2 用户态文件系统	13
6.3 用户空间下的内存磁盘	13
第 7 章 网络与外部设备	14
7.1 tcpdump 和 socket 管理	14
7.2 高速通信库: NCCL	14
7.3 数据平面开发套件: DPDK	14
第 8 章 综合应用	15
8.1 使用 RDMA 的远程内存 Swap	15
8.1.1 指标	15
8.1.2 快速上手和实现重点提示	15
8.2 用户程序嵌入内核执行	16
8.2.1 指标	16
8.2.2 快速上手和实现重点提示	16
8.3 内核空间下的内存磁盘	17
8.3.1 指标	17
8.3.2 快速上手和实现重点提示	17
8.4 机密虚拟机 Trapless 的宿主协同内存管理	17
8.4.1 指标	17
8.4.2 快速上手和实现重点提示	18
参考文献	19

实践任务的背景

在 21 世纪 20 年代，机器学习的极具发展、大模型的快速迭代使得操作系统相关的内容成为“不受大家待见”的东西。学术界急速转向机器学习系统相关系统使得大家对于传统硬核的操作系统失去了兴趣。不仅班级内发生了转向，大多数实验室也在这一期间放弃原有传统系统研究而转向新兴与大模型结合的系统研究。为了适应这一潮流，对于对计算机系统没有兴趣的同学，助教们经过讨论认为需要将重点放在“如何使用好操作系统”和“如何理解操作系统的能力”两个部分。为此，助教们结合 2019、2020、2021、2022 级日常作业的内容，将小作业实践总结的目标进行整合，整理出了这一版实践的目标手册。

由于时间仓促，这一版本内容相对比较单薄。期待同学们在作业发展的过程中不断迭代增加内容。

第一版实践手册牵头讨论和实践的同学是郑文鑫（2017）、王鲲鹏（2022）、房诗涵（2022）、徐子绎（2022）、潘屹（2022）。

第 1 章 总述

1.1 作业内容

本系列作业旨在引导大家通过修改 Linux 内核的实现感受操作系统的功能。作业具体包括以下 6 个大类：启动、系统调用与特权态执行、内存管理、文件系统、网络与外部设备、安全。每个大类进一步根据难度和内容分为基础、实践、设计、综合。前三部分的内容如表 1.1 所示。综合部分的选题每一个选题横跨两个子项目，内容和目标如表 1.2 所示。

表 1.1: 分项内容

分项	基础	实践	设计
系统启动	Read ACPI Table	Hack ACPI Table	UEFI 运行时服务
系统调用 特权态	用户定义的系统调用	vDSO	无需中断的系统调用
内存管理	页表和文件页	内存文件系统	内存压力导向的内存管理
文件系统	inode 和扩展属性管理	FUSE	用户空间下的内存磁盘
网络 外部设备	tcpdump 和 socket 管理	NCCL	DPDK

表 1.2: 综合部分分项内容

项目	涉及分项
使用 RDMA 的远程内存 Swap	内存管理、网络与外部设备
用户程序嵌入内核执行	系统调用、内存管理
内核空间下的内存磁盘	内存管理、文件系统
机密虚拟机 Trapless 的宿主协同内存管理	内存管理、安全与虚拟化

1.2 作业安排（初步）

在本学期的作业中，我们将会定期安排大家进行习题课完成答疑和基本环境配置。小作业的初步计分规则如下（满分 100 分），正式发布在 3 月 1 日。

1. 基础部分：每个分项 6 分，共 30 分，必选；
2. 实践部分：每个分项 12 分，共 60 分；
3. 设计部分：每个分项 16 分，共 80 分；
4. 综合部分：每个分项 50 分，选一个共 50 分。

作业层级之间存在关联，即实践部分的内容可能会依赖基础部分的内容，设计部分的内容可能会依赖实践部分的内容，综合部分的内容可能会依赖设计部分的内容。因此，建议大家按照顺序完成作业。分数的设定同样存在层级关联，实践部分需要基础部分完全完成才可以计算，设计部分需要至少完成 2 个实践部分才可以计算，综合部分需要至少完成 1 个设计部分才可以计算。综合部分可能需要用到特别的硬件，如果需要使用特殊的硬件请联系助教。

诚信原则 如果出现以下情况，将会扣分：

1. 某一项作业与他人完全一致（即使是两人都是用大模型生成）：计小作业 0 分。
2. 某一项作业疑似使用大模型生成且没有对代码的合理解释：计该项作业 0 分。

作业提交 为了方便大家更加使用符合自己使用习惯的内核，我们允许使用任何 Linux 5.x.x 的内核版本完成作业。（不建议使用 6.0 以及更高版本，也不建议使用 5.0 之前的 4.x 版本）。请在 4 月 15 日前提交你的基础版本

号。对于每一个作业，你需要提交按照作业要求上载明的内容（部分的作业子项要求报告）和对应与 base 版本的 diff 文件。提交的文件格式为 pdf 和 patch 文件。所有作业的截止时间均为本学期 16 周周日晚上 23:59，但是自第 8 周开始，你可以提前提交作业（Code Review 仍然安排在 16 周），提前提交作业不会有额外的加分。

Code Review Code Review 将会在 16 周进行，具体时间和地点将会在之后公布。Code Review 的内容将会包括对你的代码的评价和对你的报告的评价。Code Review 的分数将会占到你分项的 30%。**你不应当认为 Code Review 是容易获得满分的。**

额外加分 我们鼓励大家通过向本 Repo 提交 PR 来提高你的分数。每一个 PR 将会有 0.1 至 2 分不等的加分（加小作业部分分），分数取决于表述的质量和作用。PR 的内容可以是对本 Repo 的文档的修正，对作业的补充，对作业的环境配置提供的建议等等。修正错别字、表述错误至多可以提交 10 次，每次 0.1 分。对作业描述的补充、环境配置按照书写的质量给分。其余类型的 PR 提交后请联系助教确认加分。要求释疑很大概率不会获得加分。重复提交不给分（重复修复同一个错别字、重复给出相似的作业描述补充等），按照 PR 的编号决定加分对象。加分给编号较小的人。最终加分不超过 20 分。

第2章 项目环境准备

2.1 基准代码

本系列的基准代码规则如下：

1. **Linux**: 任何体系结构下高于 5.10.20 版本的 Linux 内核都是可行的选项，你可以从<https://www.kernel.org/>中选择合适的版本。对于 `initramfs`，你可以选择任何一个发行版。如果你选择直接使用 `qcow` 或者其他完全预制好的 Ubuntu 镜像，你需要手动在内部更换内核以确保你的代码能够生效。
2. **QEMU**: 版本 ≥ 7 。
3. **EDK2**: 跟随<https://github.com/tianocore/edk2>选择合适的版本。其中 `Ubuntu_GCC5` 并不意味着 GCC 版本必须是 5。

2.2 在 QEMU 中启动 Linux 镜像

注 简单起见你可以直接使用预制作好的 `qcow` Ubuntu 镜像作为磁盘镜像由 `qemu` 直接引导，具体教程：<https://documentation.ubuntu.com/public-images/en/latest/public-images-how-to/launch-qcow-with-qemu/>。如果你选用这个方案，你只需要确保 QEMU 正常安装即可。

注 本区域中 `.x.x` 部分请填入自己对应的版本号。

2.2.1 QEMU 安装（二选一）

2.2.1.1 软件包安装

```
sudo apt-get install qemu-system-x86
qemu-system-x86_64 --version
```

2.2.1.2 源码编译安装

```
# 安装依赖
sudo apt-get install git libglib2.0-dev libfdt-dev libpixman-1-dev zlib1g-dev ninja-build

# 下载编译
mkdir ~/kvm && cd ~/kvm
wget https://download.qemu.org/qemu-7.2.0.tar.xz
tar -xf qemu-7.2.0.tar.xz
cd qemu-7.2.0
./configure --target-list=x86_64-softmmu
make -j$(nproc)
```

2.2.1.3 常见依赖问题

- **ERROR: Cannot find Ninja** → 执行 `sudo apt install ninja-build`
- **ERROR: glib-2.56 required** → 执行 `sudo apt install libglib2.0-dev`
- **Dependency "pixman-1" not found** → 执行 `sudo apt install libpixman-1-dev`

2.2.2 Linux 内核编译

```
# 安装依赖
sudo apt-get install git fakeroot build-essential ncurses-dev xz-utils libssl-dev bc flex libelf-dev bison
mkdir ~/kernel && cd ~/kernel
wget https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux.git/snapshot/linux-5.x.x.tar.gz
tar -xf linux-5.x.x.tar.gz
cd linux-5.x.x

make defconfig # 使用默认配置
make -j$(nproc) # 开始编译
```

编译输出文件: arch/x86/boot/bzImage

在编译过程中，可能会发生编译错误的情况。这可能是 GCC 版本过高导致的。这里给出一种可行的组合：Linux-5.19.17, 使用 GCC-12 进行编译，操作如下：

```
sudo apt install gcc-12 g++-12 # 安装gcc-12
sudo update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-12 70
sudo update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-12 70 # 切换 gcc 默认版本
```

经过这些操作后，应该可以正常编译 Linux 内核了。

2.2.3 BusyBox

2.2.3.1 直接安装

直接使用你的开发环境自带的包管理器安装 busybox，然后在后面制作 initramfs 的过程中添加这两步：先把 busybox 拷贝进 initramfs：

```
cp $(which busybox) "${INITRAMFS_DIR}/bin/busybox"
```

然后在系统启动脚本里添加：

```
/bin/busybox --install -s /bin
```

2.2.3.2 手动编译安装

这里可以采用其他发行版（例如 Ubuntu 发行版等）。

```
mkdir ~/kvm && cd ~/kvm
wget https://busybox.net/downloads/busybox-1.x.x.tar.bz2
tar -xf busybox-1.x.x.tar.bz2
cd busybox-1.x.x

# 配置命令
make menuconfig
# 选中: Settings > Build Options > [*] Build static binary
# 取消: Shells > [ ] Job control

make -j$(nproc) && make install
```

如果在编译过程中出现错误（TCA_CBQ_MAX undeclared），一种解决方式是直接删除出错的“tc.c”文件。之后就可以正常编译了（版本 1.32.0 可以用此方法解决）。

2.2.4 EDK2 编译

EDK2 的可编译源码在 git submodule 里，因此相对简单可靠的方式就是直接按照官网所说的那样：

```
git clone -b stable/202408 https://github.com/tianocore/edk2.git
cd edk2
git submodule update --init
```

在编译之前，需要在系统里安装一些基础的包（例如 base-devel、uuid、nasm、acpica），然后就可以编译 BaseTools。

```
source edksetup.sh
make -C BaseTools -j$(nproc)
```

然后编辑 Conf/target.txt（参考<https://github.com/tianocore/tianocore.github.io/wiki/How-to-build-OVMF>）：

```
ACTIVE_PLATFORM = OvmfPkg/OvmfPkgIa32X64.dsc
TARGET_ARCH      = IA32 X64
TOOL_CHAIN_TAG   = GCC5
```

最后，执行：

```
build
```

然后就可以在 Build/Ovmf3264/DEBUG_GCC5/FV/目录找到 OVMF_CODE.fd 和 OVMF_VARS.fd 两个编译出的 OVMF 文件。（这里的 Ovmf3264 和 DEBUG_GCC5 和编译选项有关，也是在 Conf/target.txt 里面配置。）

2.2.5 制作 initramfs

2.2.5.1 创建启动脚本

```
cd busybox-1.x.x/_install/
mkdir -p proc sys dev tmp
echo -e '#!/bin/sh\nmount -t proc none /proc\nmount -t sysfs none /sys\nmount -t tmpfs none /tmp\nmount -t\ndevtmpfs none /dev\nexec "Hello Linux!"\nexec /bin/sh' > init
chmod +x init
```

2.2.5.2 打包文件系统

```
find . -print0 | cpio --null -ov --format=newc | gzip -9 > ../initramfs.cpio.gz
```

2.2.6 启动 QEMU

2.2.6.1 图形界面启动

```
qemu-system-x86_64 \
-kernel ./kernel/linux-5.x.x/arch/x86/boot/bzImage \
-initrd ./kvm/busybox-1.x.x/initramfs.cpio.gz \
-append "init=/init"
```

2.2.6.2 字符界面启动

```
qemu-system-x86_64 \
  -kernel ./kernel/linux-5.x.x/arch/x86/boot/bzImage \
  -initrd ./kvm/busybox-1.x.x/initramfs.cpio.gz \
  -nographic \
  -append "init=/init console=ttyS0"
```

如果使用 EDK2 的话，可以在参数里加上两行：

```
-drive if=pflash,format=raw,unit=0,file="{OVMF_CODE}",readonly=on \
-drive if=pflash,format=raw,unit=1,file="{PLAYGROUND_DIR}/OVMF_VARS.fd" \
```

2.2.7 验证

- 检查内核版本: `uname -a`
- 查看启动参数: `cat /proc/cmdline`
- 测试基本命令: `ls, mount`

2.3 Linux 环境部署 EDK2 开发环境

python 的版本要求 ≥ 3.11 ,

2.3.0.1 下载源码

```
git clone https://github.com/tianocore/edk2.git
cd edk2
git submodule update --init #大概需要几分钟
cd ..
```

2.3.0.2 网上找到的可能需要安装的一堆东西

```
sudo apt-get install build-essential uuid-dev
sudo apt-get install uuid-dev nasm bison flex
sudo apt-get install libx11-dev x11proto-xext-dev libxext-dev
```

将 NASM 更新到 2.15.x 以上版本，当前的 edk2 项目需要依赖 nasm2.15 以上版本，否则编译会报错 nasm 各版本下载链接：<http://www.nasm.us/pub/nasm/releasebuilds/?C=M;O=D>

```
cd nasm-2.16
./autogen.sh
./configure
make
make install
nasm --version
```

2.3.0.3 make 一下 BaseTools

然后 cd 到 edk2/BaseTools 下

```
make
```

显示 ok 则成功。注意这里所有的文件应该是 LF，不能是 CRLF，不然会出问题。可以先检查一下。

然后到 edk2 的 conf 目录下，创建 target.txt(template 在 readme 里面) 然后修改：TARGET_ARCH = X64
TOOL_CHAIN_TAG = GCC5

2.3.0.4 编译 UEFI 模拟器和 UEFI 工程

先到 edk2 目录下，然后执行

```
source edksetup.sh  
build
```

然后到 target.txt 里面改一下：ACTIVE_PLATFORM = OvmfPkg/OvmfPkgX64.dsc

2.3.0.5 运行

首先下载一个 ubuntu 的 iso 镜像，比如 ubuntu-24.04.1-live-server-amd64.iso 然后找到自己的 OVMF.fd，一般在 edk2/Build/OvmfX64/DEBUG_GCC5/FV/OVMF.fd 下下面的 bios 就是跑一下自己编译出的 UEFI 模拟器，cdrom 就是 iso 镜像，m 4096 是内存大小，enable-kvm 是开启虚拟化加速正常来说 sudo 可以不加，m 可以不加，kvm 的指令不需要加，但是我不加的话会报错，所以加上了

```
sudo qemu-system-x86_64 -m 4096 -cdrom ~/ubuntu-24.04.1-live-server-amd64.iso -bios ~/OVMF.fd -enable-kvm
```

第3章 系统启动

计算机上电后，系统会进行一系列初始化操作完成硬件设备的逐个启动。早年这一个过程由执行每个设备上的 ROM 中存储的代码（一般称为固件 firmware）结合 BIOS 完成。随后，更加通用的 UEFI 被提出，UEFI 的全称是 Unified Extensible Firmware Interface，它是一套统一使用 C 语言编写在 OS 启动前的固件代码段。

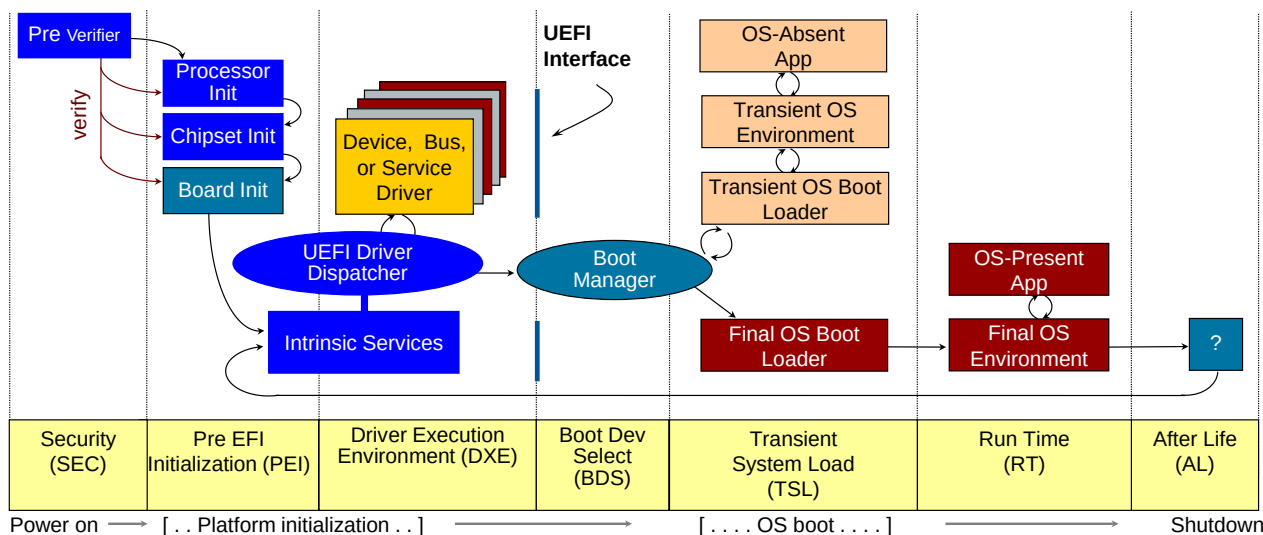


图 3.1: UEFI 启动过程

这些固件代码段还会收集硬件信息，并且移交给操作系统。早年的黑苹果、伪造 OEM 激活 Windows 7 都采用劫持 UEFI 启动后 ACPI Table 内容实现欺骗 OS 的效果。在这个专题的实践中，我们需要体验读取、修改系统启动过程中的各个表，并且尝试传递更多的信息给操作系统。

3.1 Read ACPI Table

目标：修改 EDK2 的 UEFI 组件，在其中增加函数：PrintAllACPITables。该函数不传入任何参数，要求打印每一张表的地址、长度以及每一张其中所有的信息与校验信息。

可以参考 EDK2 下的 AcpiView，但是不得直接复制。

此外，关于 ACPI 的更多信息，可以参考[这本手册](#)来学习。

具体测评指标：

- 输出所有 ACPI 表的完整地址映射；
- 每个表的输出必须包含：物理地址（64 位十六进制）、表长度（32 位十进制）、OEM ID（6 字节 ASCII）、校验和（8 位十六进制）。

3.2 Hack ACPI Table

目标：修改 EDK2 的 UEFI 组件，在其中增加函数：ChangeACPITable。该函数传入一个 UINTN 的对象表明修改第几张表、输入 EFI_ACPI_DESCRIPTION_HEADER* 表明待修改的数值，要求修改完成后打印每一张表的地址、长度以及每一张其中所有的信息与校验信息。

测试：通过引导 Linux 后读取对应的 ACPI 表检查输出进行验证。

具体评测指标：

- 修改 `EFI_ACPI_DESCRIPTION_HEADER` 中所有字段（包括 `Signature` 保留字段）且自动生成正确校验和的比例达到 100%；
- 系统重启 3 次后修改仍然有效；
- 内核日志（`dmesg | grep ACPI`）无 `CHECKSUM_ERROR` 警告或其他 `ACPI` 警告。

3.3 UEFI 运行时服务

目标：增加一个新的 UEFI 运行时服务，使得 Linux 系统可以调用这个新的服务导出硬件运行时信息。

提示：需要修改 UEFI 固件和 Linux 内核（`sysfs` 部分），方便直接读取导出的结果。

测试：展示

第 4 章 特权态与系统调用

操作系统对特权态的管理是操作系统的重要组成部分，影响了用户态程序的编写。这些系统调用也是操作系统向用户态提供抽象的重要部分，例如 Linux 中将设备的读写都转换为文件读写，因此对应的系统调用都和文件的操作读写相关。Windows 的系统调用接口则更为复杂一些，因为其内核的构成并不是单纯的宏内核，对于最小执行单元也有不同的定义。在这个专题的实践中，我们的目标是体验操作系统内核在通过系统调用进行系统管理上的重要作用。

4.1 用户定义的系统调用

目标：新增内核的键值存储模块（Key-Value Store），要求提供 read 接口和 write 接口。内核的键值数据需要存储在进程相关的结构体中，键值存储的对象是进程为对象。

提示：修改用户态系统调用的二进制文件和内核的系统调用接收器。

指标与评测要求

- 可评测指标：系统调用延迟（用户态到内核态切换时间 + 处理时间）、多线程下键值读写正确性（100 并发线程下无竞争条件）
- 吞吐要求：单核每秒可处理不少于 10,000 次空操作读写请求

实现规范

1. 接口定义：

```
// 用户态接口
int write_kv(int k, int v); // 返回写入字节数，错误返回-1
int read_kv(int k);        // 成功返回对应值，错误返回-1
```

2. 内核侧实现：

- 在进程控制块 (task_struct) 中添加 kv_store 字段：struct hlist_head kv_store[1024];
- 采用开放寻址哈希表实现，哈希函数：k % 1024
- 同步机制：每个哈希桶使用独立自旋锁 (spinlock_t)

测试方案

- 创建 100 线程交替随机读写，最终验证数据一致性
- 使用 sysbench 工具发起 100 万次随机请求
- 开启 KASAN 和 LOCKDEP 内核调试选项下测试并发正确性

4.2 vDSO

在上一个体验中，你已经增加了系统调用感受到了系统的服务。然而每次系统调用还是需要通过异常或者中断进行特权态切换，会带来较大切换开销。经过观察，可以发现一部分系统调用并不会泄露内核的状态也不会修改内核。对于这些系统调用可以在用户态以只读共享页面的方式直接进行而无需进入操作系统内核。Linux 中提供了 vDSO 支持这一想法。

目标：在 vDSO 中新增一个读取当前进程对应的 task_struct 中的所有信息的函数。

指标与评测要求

- 性能：读取数据延迟需小于 100ns（依据机器有所不同）
- 正确性：返回的 `task_struct` 指针地址与内核地址空间数据匹配

实现规范

1. 接口设计：

```
// vDSO 暴露接口
struct task_info {
    pid_t pid;
    void *task_struct_ptr;
    // 其他关键字段...
};

int get_task_struct_info(struct task_info *info);
```

2. 内核侧实现：

- 在 vDSO 镜像 (`arch/x86/entry/vdso/vdso.lds.S`) 添加新符号 `__vdso_get_task_info`
- 实现函数填充 `current` 宏指向的 `task_struct` 信息
- 用户态访问约束：所有指针字段标记为 `__user` 属性

测试方案

- 对比从 `/proc/self/stat` 获取的 `pid` 与 vDSO 返回值
- 测试多次调用 `fork()` 的接口检查稳定性

4.3 无需中断的系统调用

通过 vDSO，我们可以实现无需特权态切换对只读数据的访问。那么对于需要请求内核服务的系统调用，是否也可以避开特权态切换呢？答案是肯定的，FlexSC [3] 给出了对应的解决方案。这一解决方案可以简化为用户态程序和操作系统内核存在共享页面，用户态程序将系统调用需要的参数写进这个共享页面，然后操作系统内核从这个共享页面中读取数据并且将结果写回该页面实现基于共享内存的系统调用。

目标：在 Linux 内核中实现无特权态切换的系统调用服务。

提示：用户态线程比较容易实现请求后等待结果的自旋，需要确保内核处理的线程始终在线。

指标与评测要求

- 吞吐和延迟要求：性能应当可以提升至少 20%。
- 兼容性：需通过 Linux Test Project (LTP) 全部系统调用相关测试用例

测试方案

- 吞吐：使用 Apache Bench 进行百万次 `getpid()` 调用测试
- 正确性：运行所有 `syscall/` 目录下的 LTP 测试用例

第5章 内存管理

在计算机系统(1)中,我们在虚拟存储器部分引出了交换空间的概念。管理内存是硬件和软件协同的结果。在CPU启动后,内存的地址空间完整提供给第一个启动的软件,也就是操作系统。随后操作系统在创建每一个进程时分配页表进行页映射,从而提供每个进程完整虚拟地址空间的抽象。在这个专题的实践中,我们的目标是体验操作系统中不同的内存类型和对应管理内存的方式。

5.1 页表和文件页

目标: 读取页表,并且手动修改页表页的映射实现共享(通过 `mmap` 调整映射,通过修改内核读取页表);设置文件页实现读写落在文件的指定区域中。

测试: 测试页表映射和修改是否生效到文件中。

5.2 内存文件系统

Linux 中提供了内存文件系统(RAMfs)的支持,其有MMU版本和NoMMU版本,分别位于 `fs/ramfs/file-mmu.c` 和 `fs/ramfs/file-nommu.c`。RAMfs无法持久化,在系统崩溃时文件会全部丢失。现在你需要为RAMfs提供简单的持久化支持,当某个文件被flush到RAMfs时,该文件同步刷到一个预先设置的同步目录中以持久化该文件。

目标: 增加flush下的文件持久化。

测试: 测试持久化的文件内容语义和多线程持久化。

5.3 内存压力导向的内存管理

MacOS 给出内存压力的概念,相比于UNIX的swap大小,它被认为是更好的表达虚拟内存健康程度的标志。

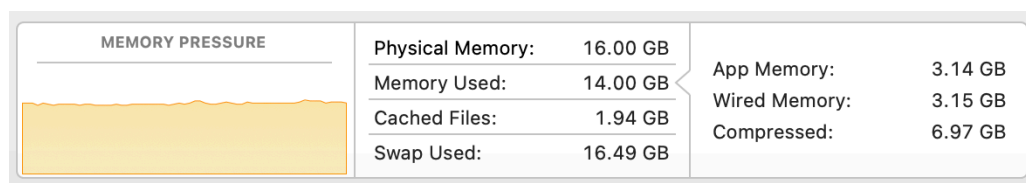


图 5.1: MacOS 中的内存压力

目标: 该实践任务包含两个子任务点。

1. 利用Linux的Wired Memory(即Active Memory)和压缩内存给出一个Linux下的内核内存压力指示器。
2. 在用户态划出一块内存空间,实现一个简单的基于内存压力的交换策略。

提示: 区分wired memory和inactive memory的区别。

测试: 测试内存压力和感知准确度的对比(演示),测试交换策略。

第 6 章 文件系统

文件是 UNIX 设计哲学中的一个重要组成部分，一切皆文件的抽象使得用户调整系统的参数也变得更加简单。因此，如何实现一个既能管理系统又能管理日常文件的文件系统便显得更加重要。Linux 通过一套统一的 VFS（虚拟文件系统）抽象达成这一目标。在本实践部分，你需要对 VFS 进行修改并且尝试使用文件系统。

6.1 inode 和扩展属性管理

目标：读取设置 inode 的基本信息，引入新的文件扩展属性 (Extended file attributes)。

测试：文件扩展属性的读写。

6.2 用户态文件系统

由于文件系统在 Linux 中被编译为内核对象 (kernel object)，文件系统崩溃会传导到内核造成内核不可用。此外，内核对象的灵活性较差。Linux 给出了 FUSE 的服务，允许在用户态实现一个文件系统。

目标：使用 FUSE 实现 GPT 服务。声明一个 GPTfs，其中的目录为每一个对话 session，对话 session 文件夹下的 input 为本轮用户输入的 prompt，output 文件为 GPT 输出的结果。每一轮对话都是单轮对话，无需考虑记录上下文。

测试：基本的读写效果

6.3 用户空间下的内存磁盘

随着硬件的发展，内存的价格已经显著下降。内存的访问速度非常快，可以作为一部分反复读写的文件的临时存储。在这一个任务中，你需要从用户态划出一块区域用于磁盘存储。

目标：使用 FUSE 实现 RAMfs 功能，要求读写碎片尽可能少并且支持硬链接。

提示：可以参考内核中 RAMfs 的功能，但是不要抄袭。

测试：文件读写和并发读写。

第7章 网络与外部设备

7.1 tcpdump 和 socket 管理

Linux 内核中提供了抓包等服务作为内核的网络套件的一部分。

目标：实现如下内容

1. 按照一定条件进行抓包（要求修改 tcpdump，不允许直接用现成的参数）
2. 内核管理 socket 的安全性实现 per-thread 的 socket fairness 管理。

测试：多 Socket 读写的管理（测试 p99 延迟）

7.2 高速通信库：NCCL

现代的高性能计算已经不能由单节点计算满足，需要多个节点多台机器进行并行计算。随着单节点算力的不断上升，并行计算时的通信成为了影响整个计算集群效率的关键因素。传统的网络通信已经不能满足这一通信需求，各个计算卡的厂商普遍提供了自己的高速互联协议（如英伟达的 NCCL、华为的 HCCL）。

目标：通过 NCCL 实现简单的信息传输，比较和传统使用以太网的速度、正确性差异。

7.3 数据平面开发套件：DPDK

当前的网络协议都在内核空间中，因此涉及网络的操作都需要通过特权切换进入内核。对于需要高速通信的程序，反复的上下文切换显然不符合要求。不过，由于完整实现基于 DPDK 的通信所需要的代码量太大，我们在这里进行简化，只需要实现简单的网络即可。

目标：实现用户态基于 DPDK 的网络 QoS 管理，实现多个进程访问网络的带宽平衡。

第8章 综合应用

当你来到这一章节时，你应该对 Linux 内核的主要组成部分有了清晰的了解，对每个部分的修改效果应该也熟悉了。在这个基础上，我们进一步探讨我们的综合应用。

8.1 使用 RDMA 的远程内存 Swap

在这个项目中，你需要实现一个简单的使用远程内存的交换分区（swap）。具体而言，当缺页异常发生时，你需要使用 RDMA 访问远程内存将页面换到本地。如果远程页面不存在，你需要寻找本地交换文件中寻找这个目标页面。在两者都不存在的情况下，应当给出一个内存错误（如同访问非法地址的错误）。

8.1.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将交换分区的内存页面通过 RDMA 存储到远程内存上。
2. 设定一个可调整的比例决定远程内存和本地内存的比例。
3. 对应用无感。

基础评测指标。 项目的基础评测指标如下：

1. 使用 memcached [2] 存储一系列数据（YCSB [1]）在使用 50% 远程内存时在 50% 能够产生一个显著的时间差。
2. 使用 RDMA 的交换时间需要比磁盘交换更短。

对于基础评测指标，一个简单的示意图如图 8.1 所示。

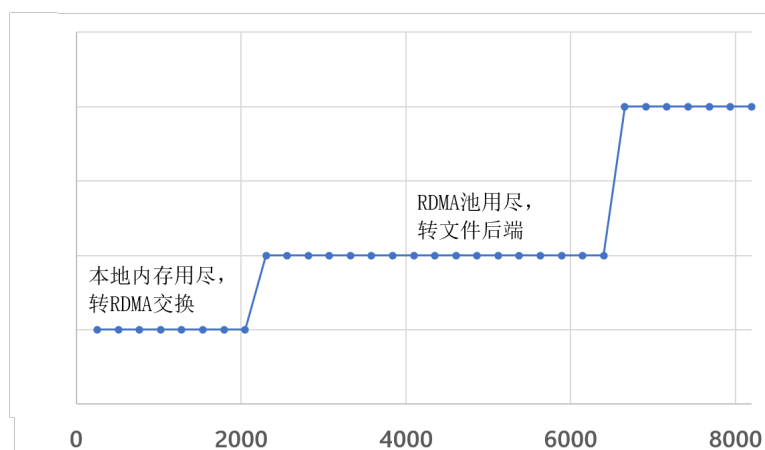


图 8.1: RDMA 和文件混合交换分区下的内存访问量同时间的示意图。图中假设本地内存为 2048MB，RDMA 内存约 4096MB。相对时间仅为示意图，不代表真实场景数据。

进阶指标。 在基础评测的指标上，结合内存访问的特点，实现页预取和交换，目标发生尽可能少的缺页异常。

8.1.2 快速上手和实现重点提示

本项目的核心包括两个部分，内存管理和 RDMA 的访问。对于内存管理部分，需要修改原有系统的 Page Fault 处理流程。RDMA 部分则需要维护 RDMA 连接确保发生缺页异常时向远端设备发起 RDMA 读写请求。

内存管理。 缺页异常发生后，操作系统需要确认这个页位于什么地方（是在内存中但是页表没有映射？还是这个页已经被交换到磁盘设备？或是这个页根本不存在？）。传统的顺序流程可以在体系结构下的 `mm/fault.c` 中找到相关的代码。加入 RDMA 后，需要保证 RDMA 部分不能被换出，并且加入对 RDMA 内存的搜索。

RDMA 管理。 RDMA 的通信与不同的以太网通信不同。为了保证传输的速度，RDMA 没有使用套接字 (Socket) 抽象。取而代之的是两大类 Verbs (Memory Verbs 和 Message Verbs) 这两大类的 Verbs 具体包含：Read (远程主机读取部分内存)、Write (数据写到远端主机)、Atomic (原子操作)；Send (把数据发送到远程 QP 的接收队列里)、Receive (接收主机被告知接收到数据缓冲)。具体的管理可以参考以下链接：(中文) <https://zhuanlan.zhihu.com/p/649468433> (英语官方原版) <https://academy.nvidia.com/en/course/rdma-programming-intro/?cm=446>

特殊点提示。

- 多线程的访问安全性：多个进程同时发生缺页异常在远端的处理过程；
- RDMA 内存的管理：减少远端内存中的内存碎片。

8.2 用户程序嵌入内核执行

用户程序执行时需要通过系统调用 (syscall) 调用特权系统服务，每一次系统调用都有上下文切换的开销，从而使得密集调用系统调用的程序产生额外的开销。一种简单的优化方法是将含有系统调用的程序嵌入内核执行，从而避免上下文切换。这一更改实际上给内核打了很多的“洞”，忽略了内核的权限隔离，因此是不安全的。我们深入之后会发现，这一想法的核心其实是一部分上下文权限隔离对程序而言过强了，程序没有用到对应的功能因此也不需要对应的隔离。在这个基础上，我们可以通过为一些简单程序设计内核中隔离他们程序的方案。具体而言，通过修改 fork 的过程将程序直接嵌入内核空间，增加一些特殊的机制（你自定义的机制，可以是页表、MPK 等）实现该段代码在内核中仅能访问特定内存（相当于自己的地址空间）。假设程序不是恶意的，因此不需要你考虑控制流完整性，但是需要注意代码的映射区域。同时假设在这项作业中程序已经被编译为位置无关代码。

8.2.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将一段程序嵌入到内核态执行，程序本身无需修改。
2. 程序仅能访问自己的内存区间，而不能访问内核的其他数据结构。

基础评测指标。 项目的基础评测指标如下：

1. 一段仅包含一系列系统调用的程序可以进入内核实现吞吐的显著提升。
2. 程序访问非法内存应当退出当前程序的执行，给出错误信息（可以在内核日志）并且不影响当前内核的执行。
3. 支持多线程程序的执行，不影响多线程程序的可扩展性。

进阶指标。 在基础评测的指标上，支持带有多个用户库文件的用户程序进入内核执行。

8.2.2 快速上手和实现重点提示

本项目的核心包括两个部分，代码启动阶段 (fork) 和执行阶段 (exec)。

启动阶段 启动阶段用户程序并没有执行，核心需要考虑的部分是如何映射用户段的代码到内核空间，以及如何设置权限隔离的初始状态。此外，需要在启动阶段拦截系统调用，确保系统调用转换为内核态的函数跳转。

执行阶段 执行阶段最核心的是在遇到错误的时候如何修正错误的状态，从而避免嵌入内核的程序崩溃内核。

特殊点提示。

- 多线程执行：与内核线程的绑定执行关系；
- 内核状态的保护：防止单个程序崩溃整个内核。

8.3 内核空间下的内存磁盘

随着硬件的发展，内存的价格已经显著下降。内存的访问速度非常快，可以作为一部分反复读写的文件的临时存储。这一部分是用户空间下的内存磁盘的进一步延伸。

8.3.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 将一段内存空间作为磁盘空间映射给内核。
2. 映射的磁盘空间可以进行文件读写和文件夹创建。
3. 注册为一个磁盘设备而非文件系统。

基础评测指标。 项目的基础评测指标如下：

1. 使用 `dd` 一次性创建一个文件的开销足够小（需要接近内存的读写速度）。
2. 多次创建删除文件之后实际存储容量始终保持一致。

进阶指标。 在基础评测的指标上，减小文件元数据的开销大小。

8.3.2 快速上手和实现重点提示

本项目的核心是内存空间的映射和分区管理。与映射在用户空间不同，内核空间的映射需要保证用于虚拟磁盘的数据不能破坏内核数据的正确性。此外，用户空间可以通过 `fuse` 实现用户态的分区访问。在内核中，该模块需要实现为内核模块，因此需要符合内核的一系列数据接口。

特殊点提示。

- 多线程执行：多线程读写保证数据的一致性。
- 文件碎片：文件的碎片应该尽可能少。

8.4 机密虚拟机 Trapless 的宿主协同内存管理

机密虚拟机的一个核心是内存需要加密以防止宿主可以获取到虚拟机内存造成数据泄露，然而这样的安全措施也造成了宿主机无法感知虚拟机内的内存情况，内存分配无法最优化。一种方式是虚拟机内核主动向宿主机的虚拟机管理器发起调用引导宿主机按照一定的偏好管理内存。这种方式需要下陷到宿主机的虚拟机管理器，有较大的切换开销。请设计一种机制，以不下陷的方式引导宿主机的按照一定偏好管理内存。

8.4.1 指标

项目形态。 你的项目最终的形态应当符合以下指标：

1. 不修改虚拟机内应用程序，允许修改虚拟机管理器和虚拟机内的内核。
2. 指示内存偏好时不发生下陷。

基础评测指标。 项目的基础评测指标如下：

1. 使用内存密集型任务在机密虚拟机内部设置特定内存位置后计算速度和用户态的读写速度一致。

8.4.2 快速上手和实现重点提示

本项目的指标较少，因为核心代码需要较大的修改，包括三个部分：宿主机的内核、虚拟机的内核、宿主机的虚拟机管理器。宿主机内核需要设定特殊的接口管理加密的内存页面。虚拟机内核需要修改实现无下陷与宿主机的虚拟机管理器进行交互。宿主机的虚拟机内存管理器需要增加接收虚拟机内核的特殊内存偏好参数并且传递给宿主机。

特殊点提示。

- 虚拟机下陷的处理。
- 共享内存页的管理。
- 宿主机的内存管理。

参考文献

- [1] Brian F Cooper et al. “Benchmarking cloud serving systems with YCSB”. In: *Proceedings of the 1st ACM symposium on Cloud computing*. 2010, pp. 143–154.
- [2] Brad Fitzpatrick. “Distributed caching with memcached”. In: *Linux journal* 2004.124 (2004), p. 5.
- [3] Livio Soares and Michael Stumm. “{FlexSC}: Flexible system call scheduling with {Exception-Less} system calls”. In: *9th USENIX Symposium on Operating Systems Design and Implementation (OSDI 10)*. 2010.